IBM HR Analytics Employee Attrition & Performance

In [1]:
```
# importing libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# to display plots and graphs inline
%matplotlib inline
```

In [2]:
```
# using the seaborn style for graphs
plt.style.use("seaborn-v0_8")
```

In [3]:
```
# read the dataset
employee_data = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")
```

In [4]:
```
employee_data.shape
```

Out[4]:
```
(1470, 35)
```

In [5]:
```
employee_data.head()
```

Out[5]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educa
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life
4	27	No	Travel_Rarely	591	Research & Development	2	1	

5 rows × 35 columns

In [6]:
```
# finding missing values
employee_data.isnull().sum()
```

Out[6]:

```
Age                         0
Attrition                   0
BusinessTravel              0
DailyRate                   0
Department                  0
DistanceFromHome            0
Education                   0
EducationField              0
EmployeeCount               0
EmployeeNumber              0
EnvironmentSatisfaction     0
Gender                      0
HourlyRate                  0
JobInvolvement              0
JobLevel                    0
JobRole                     0
JobSatisfaction             0
MaritalStatus               0
MonthlyIncome               0
MonthlyRate                 0
NumCompaniesWorked          0
Over18                      0
OverTime                    0
PercentSalaryHike           0
PerformanceRating           0
RelationshipSatisfaction    0
StandardHours               0
StockOptionLevel            0
TotalWorkingYears           0
TrainingTimesLastYear       0
WorkLifeBalance             0
YearsAtCompany              0
YearsInCurrentRole          0
YearsSinceLastPromotion     0
YearsWithCurrManager        0
dtype: int64
```

In [7]:

```
#checking data types
employee_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64
 6   Education                 1470 non-null   int64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64
 9   EmployeeNumber            1470 non-null   int64
 10  EnvironmentSatisfaction   1470 non-null   int64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

Exploratory Data Analysis

In [8]:
```
# basic descriptive statistics
employee_data.describe().T
```

Out[8]:

	count	mean	std	min	25%	50%	75%	
Age	1470.0	36.923810	9.135373	18.0	30.00	36.0	43.00	
DailyRate	1470.0	802.485714	403.509100	102.0	465.00	802.0	1157.00	1
DistanceFromHome	1470.0	9.192517	8.106864	1.0	2.00	7.0	14.00	
Education	1470.0	2.912925	1.024165	1.0	2.00	3.0	4.00	
EmployeeCount	1470.0	1.000000	0.000000	1.0	1.00	1.0	1.00	
EmployeeNumber	1470.0	1024.865306	602.024335	1.0	491.25	1020.5	1555.75	2
EnvironmentSatisfaction	1470.0	2.721769	1.093082	1.0	2.00	3.0	4.00	
HourlyRate	1470.0	65.891156	20.329428	30.0	48.00	66.0	83.75	
JobInvolvement	1470.0	2.729932	0.711561	1.0	2.00	3.0	3.00	
JobLevel	1470.0	2.063946	1.106940	1.0	1.00	2.0	3.00	
JobSatisfaction	1470.0	2.728571	1.102846	1.0	2.00	3.0	4.00	
MonthlyIncome	1470.0	6502.931293	4707.956783	1009.0	2911.00	4919.0	8379.00	19
MonthlyRate	1470.0	14313.103401	7117.786044	2094.0	8047.00	14235.5	20461.50	26
NumCompaniesWorked	1470.0	2.693197	2.498009	0.0	1.00	2.0	4.00	
PercentSalaryHike	1470.0	15.209524	3.659938	11.0	12.00	14.0	18.00	
PerformanceRating	1470.0	3.153741	0.360824	3.0	3.00	3.0	3.00	
RelationshipSatisfaction	1470.0	2.712245	1.081209	1.0	2.00	3.0	4.00	
StandardHours	1470.0	80.000000	0.000000	80.0	80.00	80.0	80.00	
StockOptionLevel	1470.0	0.793878	0.852077	0.0	0.00	1.0	1.00	
TotalWorkingYears	1470.0	11.279592	7.780782	0.0	6.00	10.0	15.00	
TrainingTimesLastYear	1470.0	2.799320	1.289271	0.0	2.00	3.0	3.00	
WorkLifeBalance	1470.0	2.761224	0.706476	1.0	2.00	3.0	3.00	
YearsAtCompany	1470.0	7.008163	6.126525	0.0	3.00	5.0	9.00	
YearsInCurrentRole	1470.0	4.229252	3.623137	0.0	2.00	3.0	7.00	
YearsSinceLastPromotion	1470.0	2.187755	3.222430	0.0	0.00	1.0	3.00	
YearsWithCurrManager	1470.0	4.123129	3.568136	0.0	2.00	3.0	7.00	

In [9]:
```
# mapping the attrition 1 => yes and 0 => no in the new column

employee_data["left"] = np.where(employee_data["Attrition"] == "Yes",1,0)
```

In [10]:
```
employee_data.head()
```

Out[10]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educa
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life
4	27	No	Travel_Rarely	591	Research & Development	2	1	

5 rows × 36 columns

In [11]:
```
# supressing all the warnings
import warnings
warnings.filterwarnings('ignore')
```

In [12]:
```
def Numerical_Variables_TargetPlots(df,segment_by,target_var ="Attrition"):
    """A function for plotting the distribution of numerical variables
     and its effect on attrition """

    fig, ax = plt.subplots(ncols= 2, figsize = (14,6))

    #boxplot for comparison
    sns.boxplot(x= target_var, y= segment_by, data=df,ax=ax[0])
    ax[0].set_title("Comparison of " + segment_by + " vs "+ target_var)

    #distribution plot
    ax[1].set_title("Distribution of "+segment_by)
    ax[1].set_ylabel("Frequency")
    sns.distplot(a= df[segment_by],ax=ax[1], kde=False)

    plt.show()
```

In [13]:
```
def Categorical_Variables_targetPlots(df, segment_by,invert_axis =False,target_var
    """A function for plotting the effect of variables(categorical data)
    on Attrition"""

    fig, ax = plt.subplots(ncols=2, figsize = (14,6))

    # countplot for distribution along with target variable
    # invert axis variable helps to interchange the axis so that
    # names of categories doesn't ovarlap
    if invert_axis == False:
        sns.countplot(x = segment_by, data=df, hue="Attrition",ax=ax[0])
    else:
        sns.countplot(y = segment_by, data=df, hue="Attrition",ax=ax[0])

    ax[0].set_title("Comparison of " + segment_by + " vs "+ "Attrition")

    #plot the effect of the variable on attrition
    if invert_axis == False:
        sns.barplot(x = segment_by, y = target_var, data=df,ci=None)
    else:
        sns.barplot(y= segment_by, x = target_var, data=df, ci=None)
```

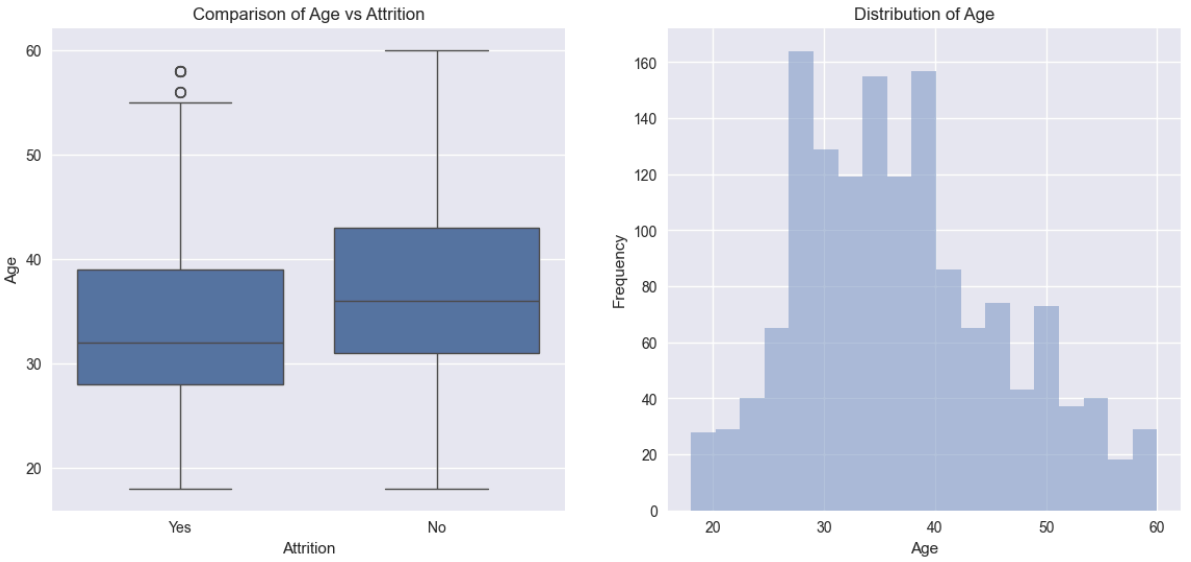
```
    ax[1].set_title("Attriton rate by {}".format(segment_by))
    ax[1].set_ylabel("Average(Attrition)")
    plt.tight_layout()

    plt.show()
```

Analyzing the variables

- Numerical Variables

Age

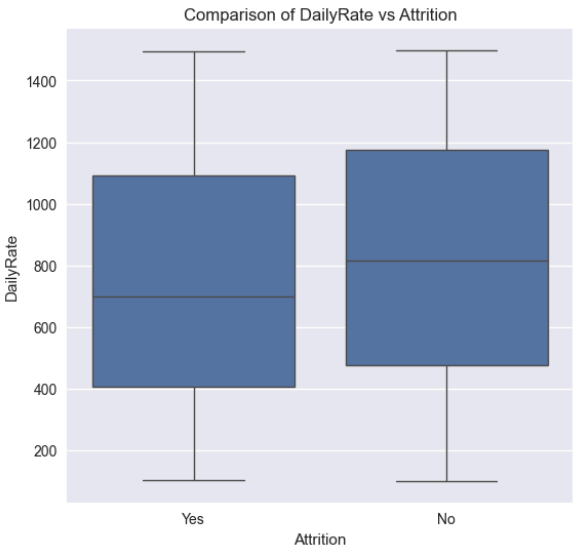
In [14]:
```
# we are checking the distribution of employee age and its related to attrition or
Numerical_Variables_TargetPlots(employee_data,segment_by="Age")
```

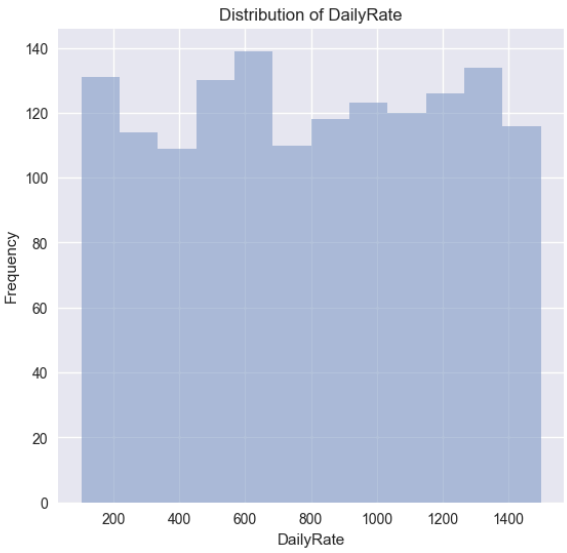


- **Age Distribution:** Employees who have stayed with the company ("No" attrition status) tend to be older on average compared to those who have left ("Yes" attrition status).
- **Median Age:** The median age for employees who have left is 30, while for those who stayed it is 37. This suggests that younger employees are more likely to leave the company.
- **Quartiles:** The age range between the first quartile and the third quartile is wider for employees who stayed, indicating a more diverse age range in the retained employees.
- **Outliers:** The presence of outliers in the "Yes" category might suggest that while the majority of younger employees are more likely to leave, there are a few older employees who also leave the company.

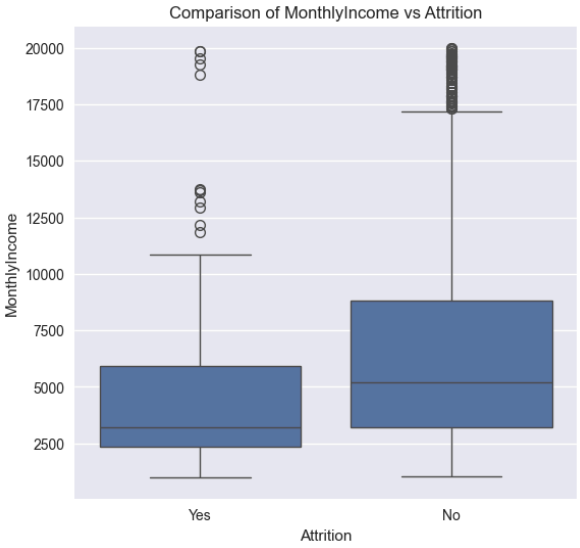
Daily Rate & Monthly Income & HourlyRate

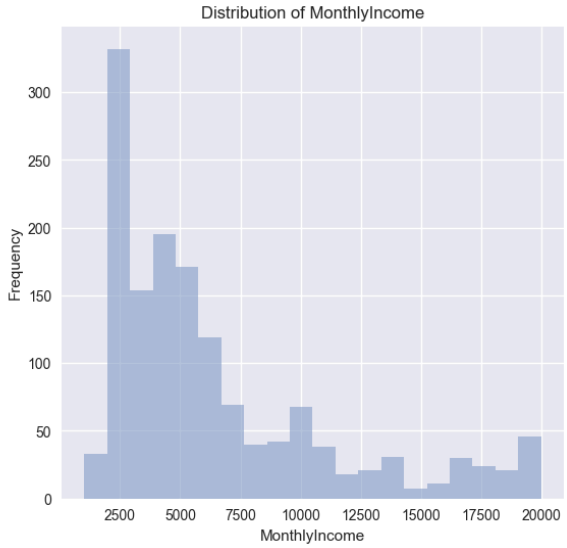
In [15]:
```
#Analyzing the daily wage rate vs employee left the company or not

Numerical_Variables_TargetPlots(employee_data,"DailyRate")
```





In [16]:
```
Numerical_Variables_TargetPlots(employee_data,"MonthlyIncome")
```

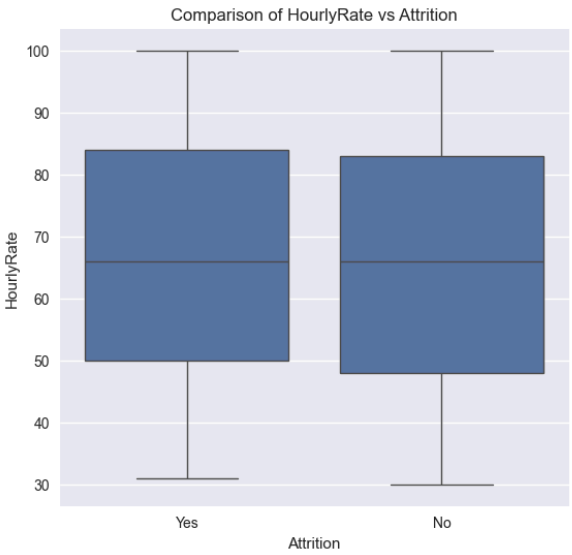


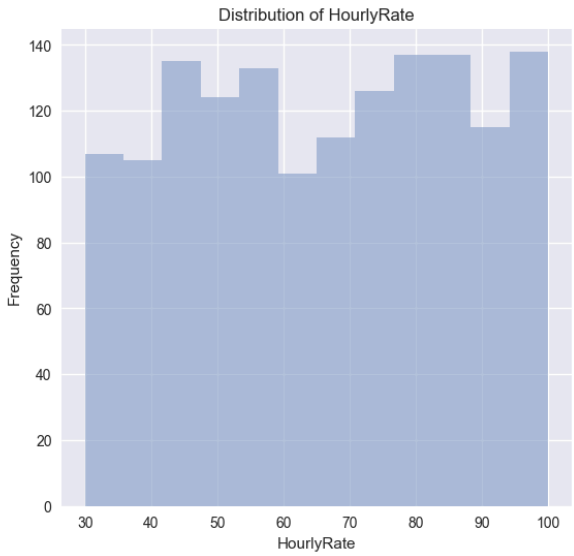


- Employee's working with lower daily rates are more prone to leave the company than compared to the employee's working with higher rates. The same trend is resonated with monthly income too.

Hourly Rate

In [17]:
```
Numerical_Variables_TargetPlots(employee_data,"HourlyRate")
```

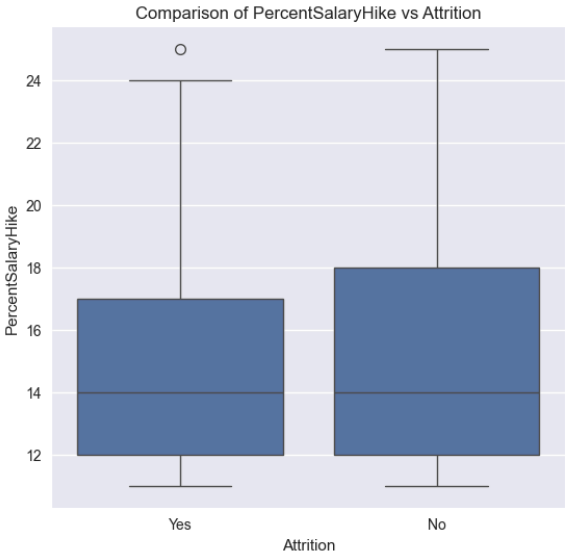


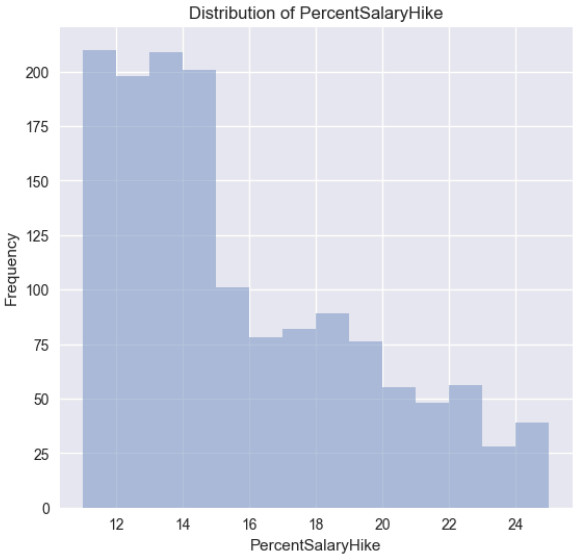


- From plot we have seen that there is no significant diffence in the hourly rate and attrition. Therefore hourly rate is considered as not significant to attrition.

Percent Salary Hike

In [18]:
```
Numerical_Variables_TargetPlots(employee_data,"PercentSalaryHike")
```

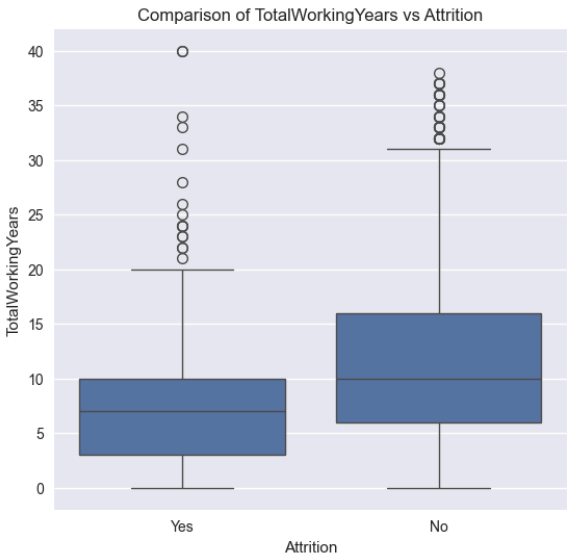


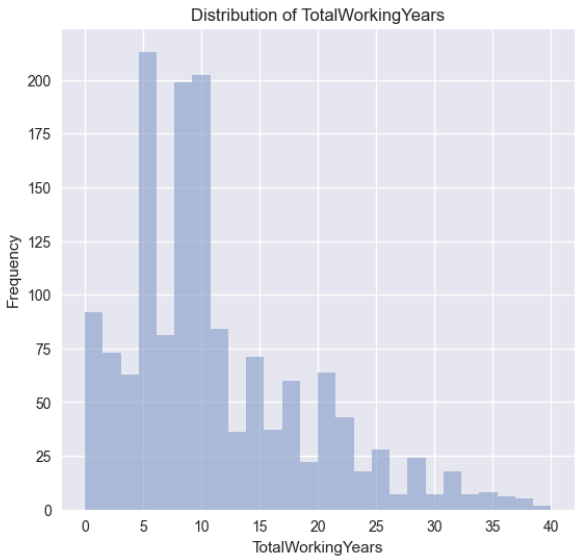


- Majority (60% of total strength) of employee's receive 16% salary hike in the company, employee's who received less salary hike have left the company.

Total Working Years

In [19]:
```
Numerical_Variables_TargetPlots(employee_data,"TotalWorkingYears")
```



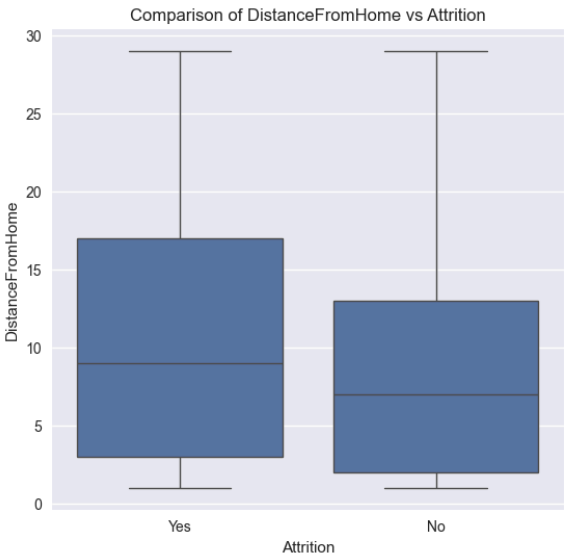


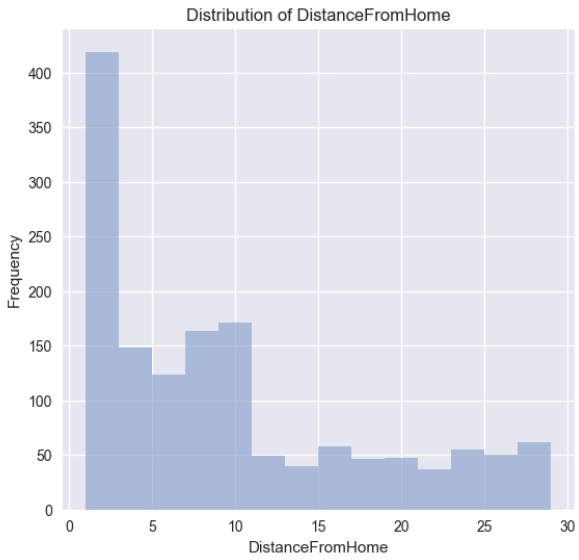
In [20]:
```
sns.lmplot(x ="TotalWorkingYears", y= "PercentSalaryHike", data=employee_data, fit_
           height=6,aspect=1.5)
plt.show()
```

- Employee's with less working years have received 25% Salary hike when they switch to another company, but there is no linear relationship between working years and salary hike.
- Attrition is not seen amomg the employee's having more than 20 years of experience if their salary hike is more than 20%, even if the salary hike is below 20% attrition rate among the employee's is very low.
- Employee's with lesser years of experience are prone to leave the company in search of better pay, irrespective of salary hike

Distance From Home

In [21]:
```
Numerical_Variables_TargetPlots(employee_data,"DistanceFromHome")
```





- There is a higher number of people who reside near to offices and hence the attrition levels are lower for distances less than 10. With increase in distance from home, attrition rate also increases

Analysing the Variables

- Categorical Variables

Job Involvement

In [22]:
```
#cross tabulation between attrition and Job Involvement
pd.crosstab(employee_data.JobInvolvement,employee_data.Attrition)
```

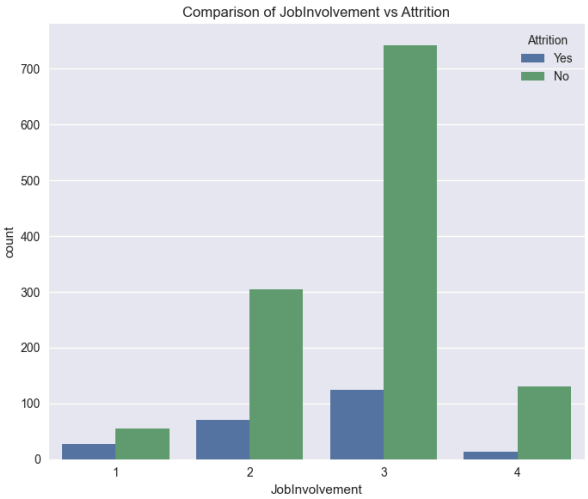
Out[22]:

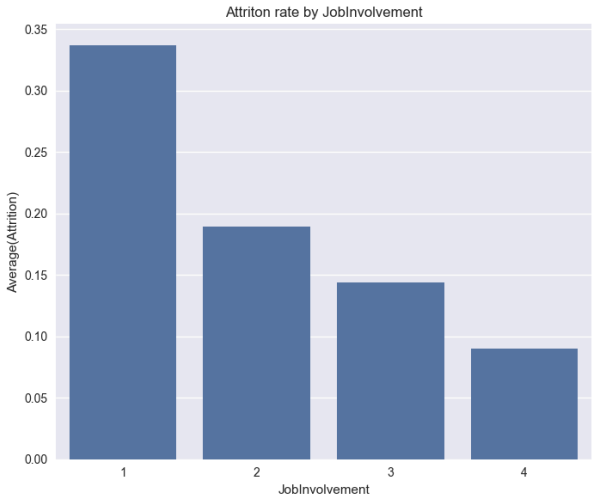
Attrition	No	Yes
JobInvolvement		
1	55	28
2	304	71
3	743	125
4	131	13

In [23]:
```
# calculate the percentage of people having different job involvement role
round(employee_data.JobInvolvement.value_counts()/employee_data.shape[0]*100,2)
```

Out[23]:
```
JobInvolvement
3    59.05
2    25.51
4     9.80
1     5.65
Name: count, dtype: float64
```

In [24]:
```
Categorical_Variables_targetPlots(employee_data,"JobInvolvement")
```

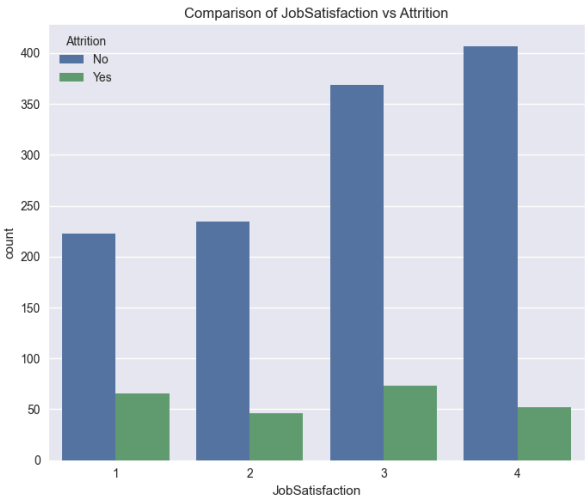


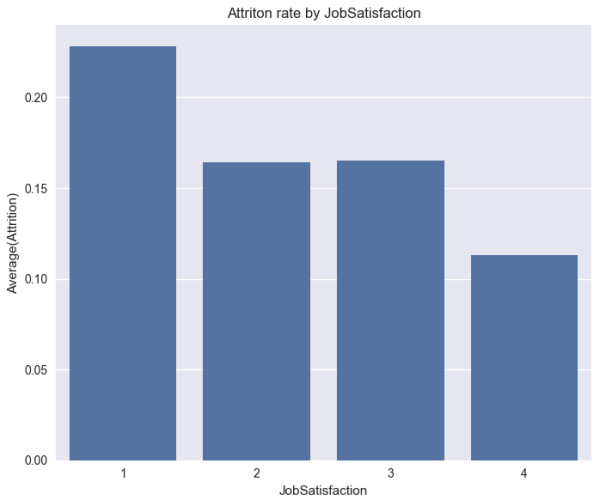


1. In the total data set, 59% have high job involvement whereas 25% have medium involvement rate
2. From above plot we can observe that round 50% of people in low job involvement (level 1 & 2) have left the company.
3. Even the people who have high job involmenent have higher attrition rate around 15% in that category have left company

Job Satisfaction

In [25]:
```
Categorical_Variables_targetPlots(employee_data,"JobSatisfaction")
```





As expected, people with low satisfaction have left the company around 23% in that category. what surprising is out of the people who rated medium and high job satisfaction around 32% has left the company. There should be some other factor which triggers their exit from the company

Performance Rating

In [26]:
```
# Checking the number of categories under performance rating
employee_data.PerformanceRating.value_counts()
```

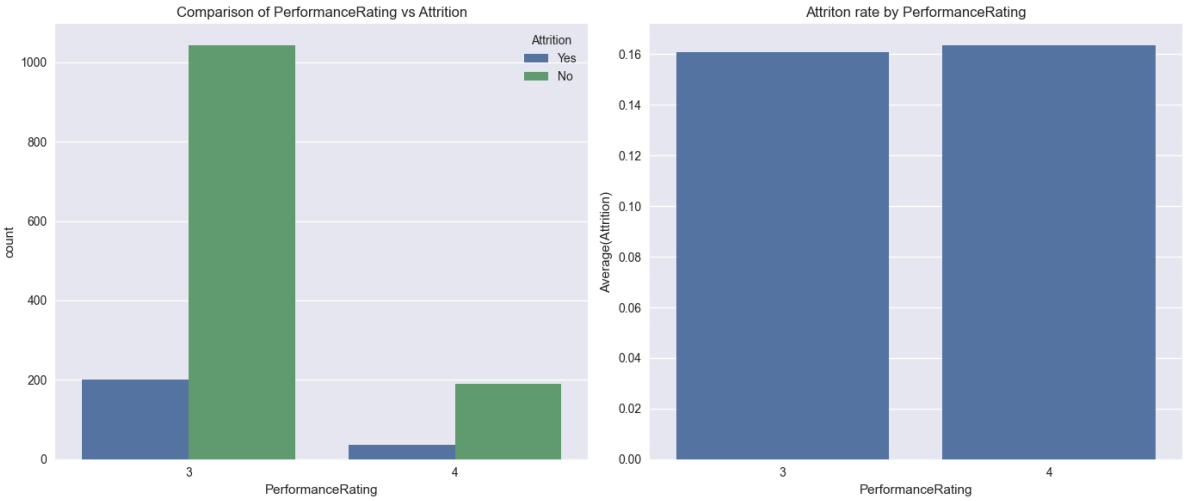
Out[26]:
```
PerformanceRating
3    1244
4     226
Name: count, dtype: int64
```

In [27]:
```
# calculate the percentage of performance rating per category in the whole dataset
round(employee_data.PerformanceRating.value_counts()/employee_data.shape[0]*100,2)
```

Out[27]:
```
PerformanceRating
3    84.63
4    15.37
Name: count, dtype: float64
```

- Around 85% of the people in the company rated as Excellent and remaining 15% rated as Outstanding

In [28]:
```
Categorical_Variables_targetPlots(employee_data,"PerformanceRating")
```



Contrary to normal beleif that employee's having higher rating will not leave the company. It may be seen that there is no significant difference between the Performance rating and Attrition rate

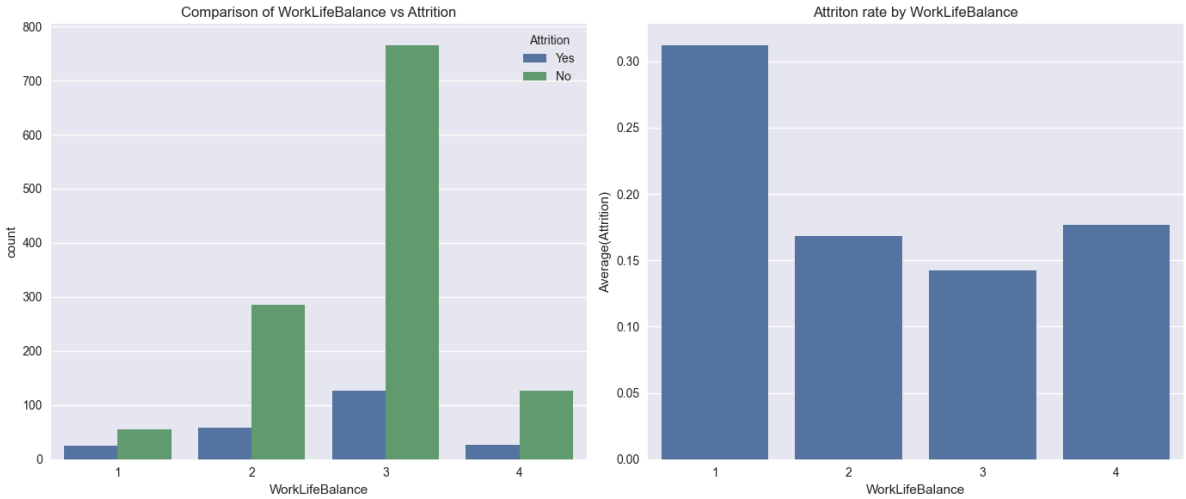
WorkLifeBalance

In [29]:
```
# percentage of worklife balance rating accross the company date
round(employee_data.WorkLifeBalance.value_counts()/employee_data.shape[0]*100,2)
```

Out[29]:
```
WorkLifeBalance
3    60.75
2    23.40
4    10.41
1     5.44
Name: count, dtype: float64
```

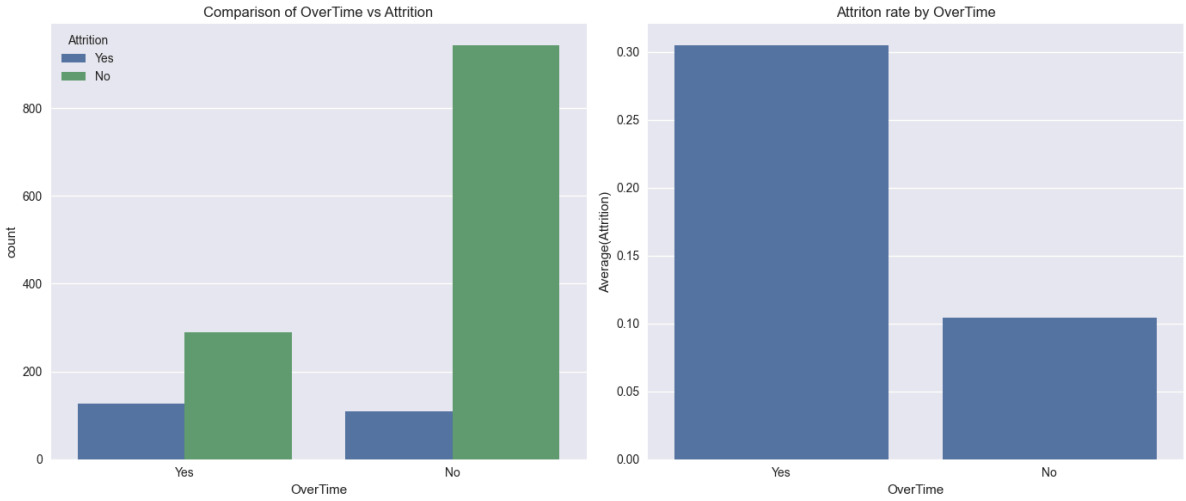
More than 60% of the employee's rated that they have Better worklife balance and 10% rated for Best worklife balance

In [30]:
```
Categorical_Variables_targetPlots(employee_data,"WorkLifeBalance")
```



- As expected more than 30% of the people who rated as Bad WorkLifeBalance have left the company and around 15% of the people who rated for Best WorkLifeBalance also left the company

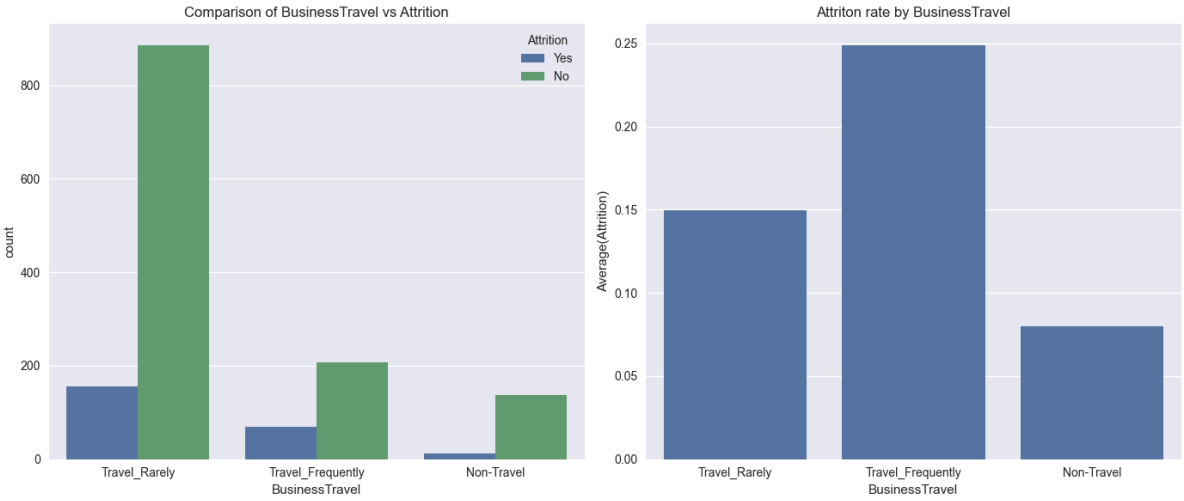
In [31]:
```
Categorical_Variables_targetPlots(employee_data,"OverTime")
```



- More than 30% of employee's who worked overtime has left the company, where as 90% of employee's who have not experienced overtime has not left the company. Therefore overtime is a strong indicator of attrition

Business Travel

In [32]:
```
Categorical_Variables_targetPlots(employee_data,segment_by="BusinessTravel")
```



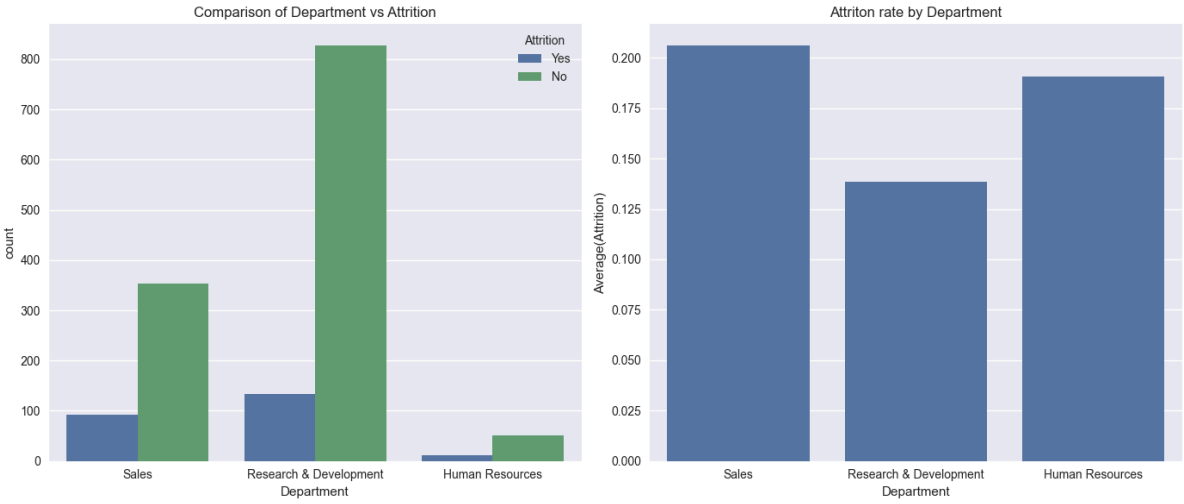
- There are more people who travel rarely compared to people who travel frequently. In case of people who travel Frequency around 25% of people have left the company and in other cases attrition rate doesn't vary significantly on travel

Department

In [33]:
```
employee_data.Department.value_counts()
```

Out[33]:
```
Department
Research & Development    961
Sales                     446
Human Resources            63
Name: count, dtype: int64
```

In [34]:
```
Categorical_Variables_targetPlots(employee_data,segment_by="Department")
```



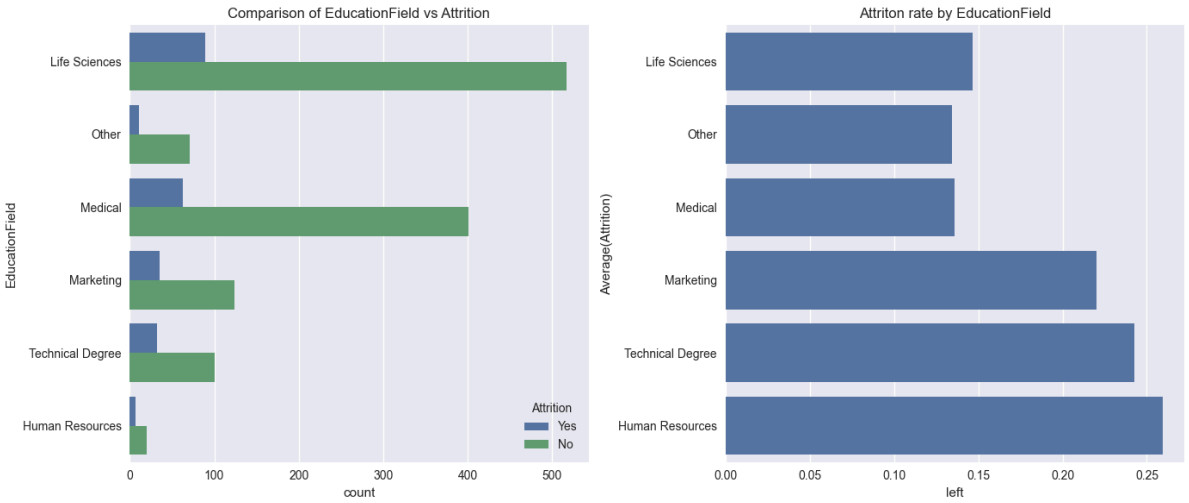
- On comparing departmentwise,we can conclude that HR has seen only a marginal high in turnover rates whereas the numbers are significant in sales department with turnover rates of 39 %. The attrition levels are not appreciable in R & D where 67 % have recorded no attrition.
- Sales has seen higher attrition levels about 20.6% followed by HR around 18%

Education Field

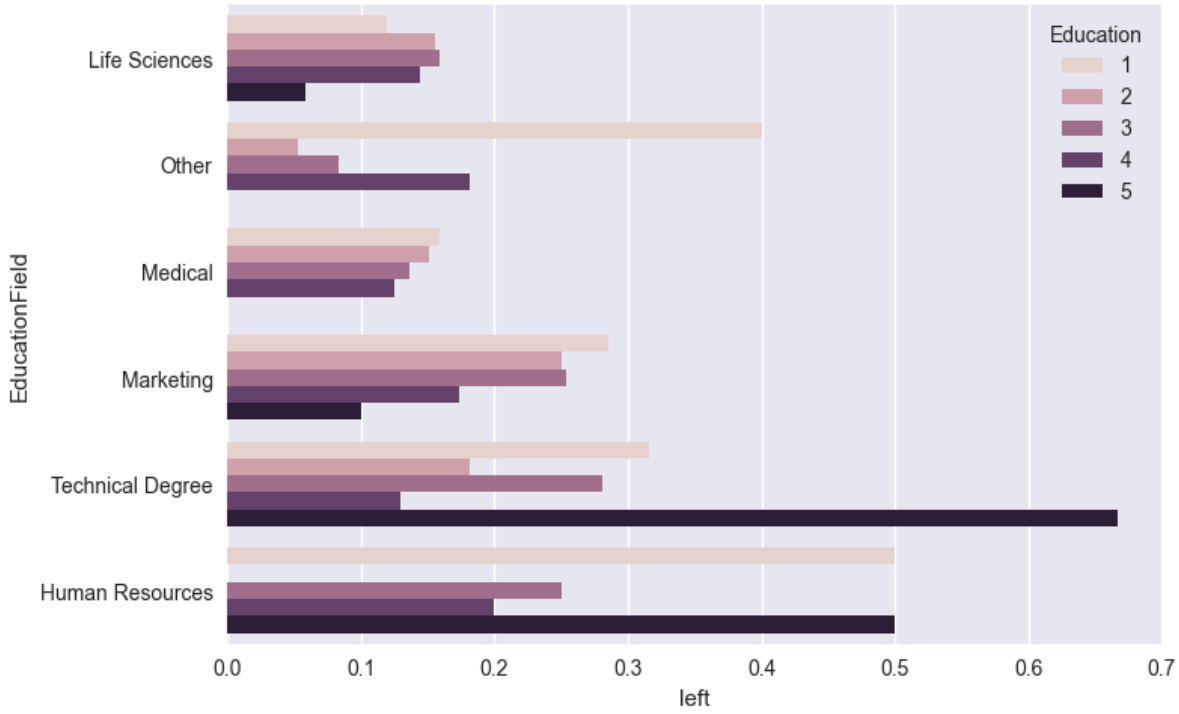
In [35]:
```
employee_data.EducationField.value_counts()
```

Out[35]:
```
EducationField
Life Sciences       606
Medical             464
Marketing           159
Technical Degree    132
Other                82
Human Resources      27
Name: count, dtype: int64
```

In [36]:
```
Categorical_Variables_targetPlots(employee_data,"EducationField",invert_axis=True)
```



In [37]:
```
plt.Figure(figsize=(10,8))
sns.barplot( y= "EducationField", x = "left", hue = "Education", data = employee_da
plt.show()
```

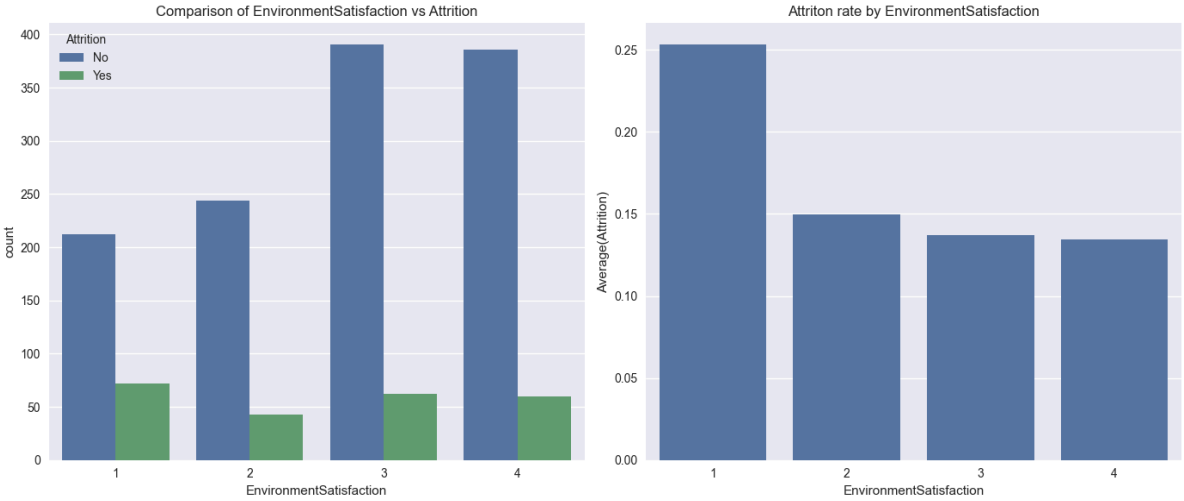


- There are more people with a Life sciences followed by medical and marketing
- Employee's in the EducationField of Human Resources and Technical Degree have highest attrition levels around 26% and 23% respectively

- When compared with Education level, we have observed that employees in the highest level of education in there field of study have left the company. We can conclude that EducationField is a strong indicator of attrition

Environment Satisfaction

In [38]:
```
Categorical_Variables_targetPlots(employee_data,"EnvironmentSatisfaction")
```

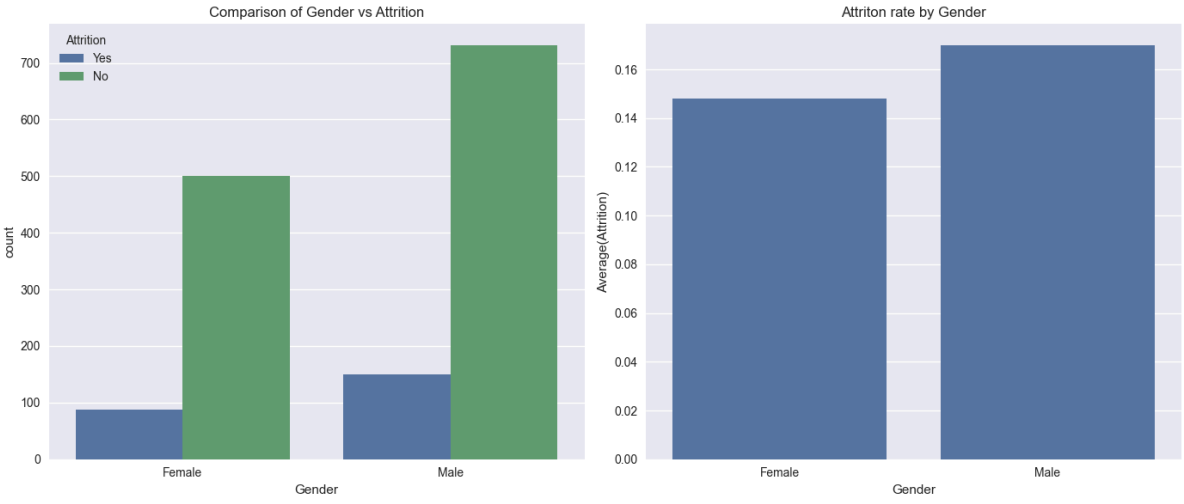


we can see that people having low environment satisfaction 25% leave the company

Gender Vs Attrition

In [39]:
```
sns.boxplot(x=employee_data['Gender'], y=employee_data["MonthlyIncome"])
plt.title("Monthly Income Vs Gender Boxplot", fontsize=20)
plt.xlabel("Gender", fontsize=16)
plt.ylabel('Monthly Income', fontsize=16)
plt.show()
```

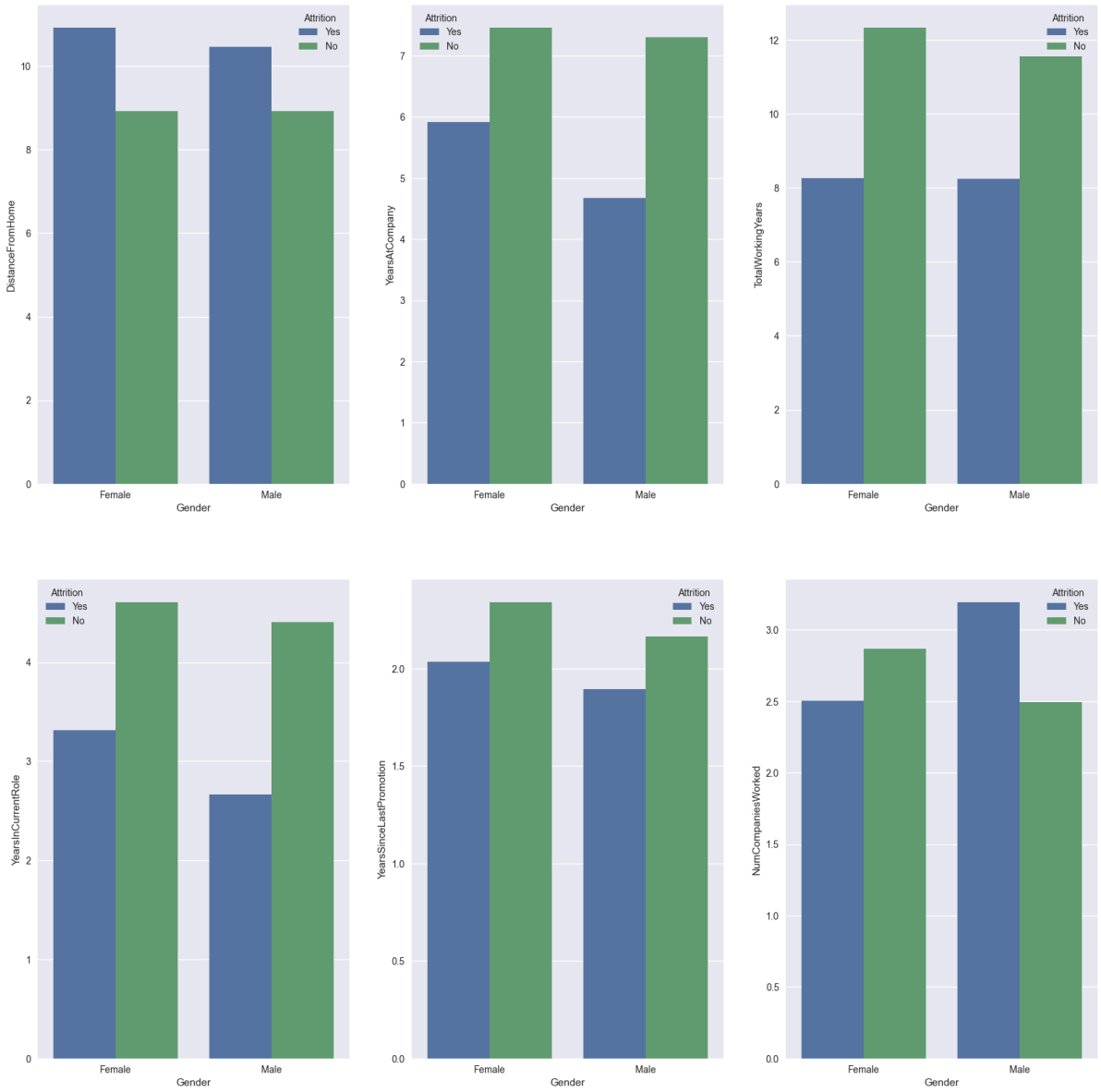
In [40]:
```
Categorical_Variables_targetPlots(employee_data,"Gender")
```



- Monthly Income distribution for Male and Female is almost similar, so the attrition rate of Male and Female is almost around 15%. Gender is not a strong indicator of attrition

In [41]:
```
fig, ax = plt.subplots(2,3, figsize=(20,20)) # "ax" has references to all the four
plt.suptitle("Comparison of various factors vs Gender", fontsize = 20)
sns.barplot(x = employee_data["Gender"], y = employee_data["DistanceFromHome"], hue
sns.barplot(x = employee_data["Gender"], y = employee_data["YearsAtCompany"], hue=
sns.barplot(x = employee_data["Gender"], y = employee_data["TotalWorkingYears"], hu
sns.barplot(x = employee_data["Gender"], y = employee_data["YearsInCurrentRole"], h
sns.barplot(x = employee_data["Gender"], y = employee_data["YearsSinceLastPromotion
sns.barplot(x = employee_data["Gender"], y = employee_data["NumCompaniesWorked"], h
plt.show()
```

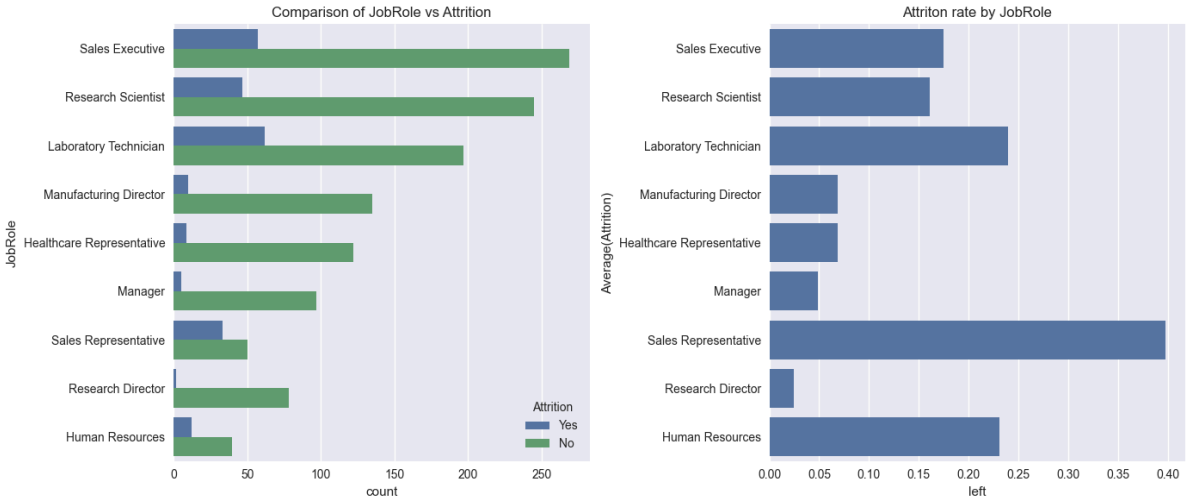
Comparison of various factors vs Gender



1. Distance from home matters to women employees more than men.
2. Female employes are spending more years in one company compare to their counterpart.
3. Female employes spending more years in current company are more inclined to switch.

Job Role

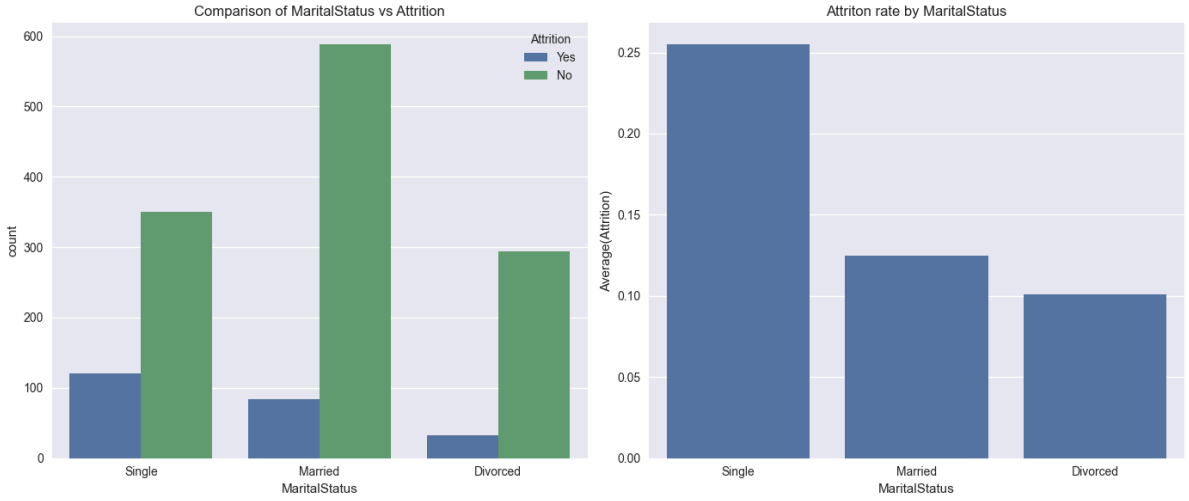
In [42]:
```
Categorical_Variables_targetPlots(employee_data, "JobRole", invert_axis=True)
```



1. Jobs held by the employee is maximum in Sales Executive, then R&D , then Laboratory Technician
2. People working in Sales department is most likely quit the company followed by Laboratory Technician and Human Resources there attrition rates are 40%, 24% and 22% respectively

Marital Status

In [43]:
```
Categorical_Variables_targetPlots(employee_data,"MaritalStatus")
```



From the plot,it is understood that irrespective of the marital status,there are large people who stay with the company and do not leave.Therefore,marital status is a weak predictor of attrition

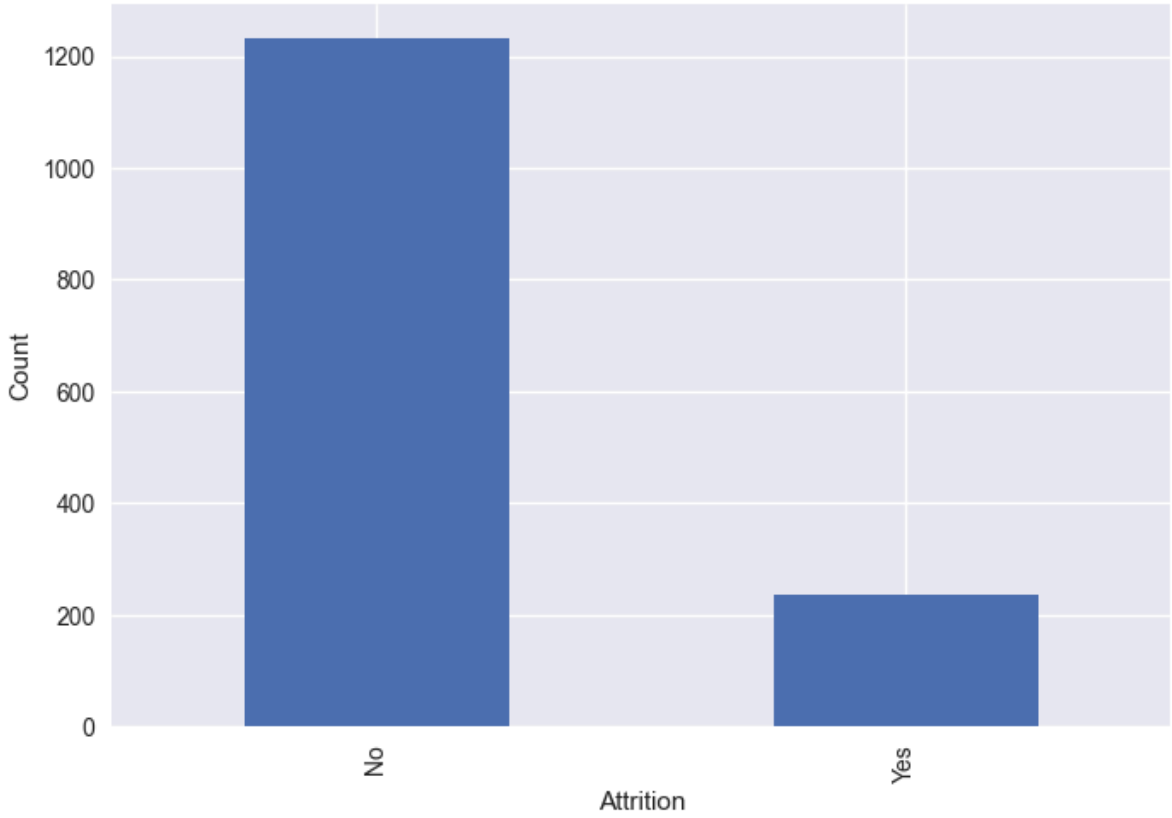
In [ ]:
```
```

Building Decision Tree

In [44]:
```
from sklearn.model_selection import train_test_split

# for fitting classification tree
from sklearn.tree import DecisionTreeClassifier

# to create a confusion matrix
```

```
from sklearn.metrics import confusion_matrix

# import whole class of metrics
from sklearn import metrics
```

In [45]:
```
employee_data.Attrition.value_counts().plot(kind="bar")
plt.xlabel("Attrition")
plt.ylabel("Count")
plt.show()
```



In [46]:
```
employee_data["Attrition"].value_counts()
```

Out[46]:
```
Attrition
No     1233
Yes     237
Name: count, dtype: int64
```

From the Exploratory data analysis, variable that are not significant to attrition are:

- EmployeeCount, EmployeeNumber, Gender, HourlyRate, JobLevel, MaritalStatus, Over18, StandardHours

In [47]:
```
# copying the main employee data to another dataframe
employee_data_new = employee_data.copy()
```

In [48]:
```
# dropping the not significant variables
employee_data_new.drop(["EmployeeCount","EmployeeNumber","Gender","HourlyRate","Ove
```

In [49]:
```
employee_data_new.head()
```

Out[49]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educa
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life
4	27	No	Travel_Rarely	591	Research & Development	2	1	

5 rows × 29 columns

Handling Categorical Variables

- Segregate the numerical and Categorical variables
- Convert Categorical variables to dummy variables

In [50]:
```
# data types of variables
dict(employee_data_new.dtypes)
```

Out[50]:
```
{'Age': dtype('int64'),
 'Attrition': dtype('O'),
 'BusinessTravel': dtype('O'),
 'DailyRate': dtype('int64'),
 'Department': dtype('O'),
 'DistanceFromHome': dtype('int64'),
 'Education': dtype('int64'),
 'EducationField': dtype('O'),
 'EnvironmentSatisfaction': dtype('int64'),
 'JobInvolvement': dtype('int64'),
 'JobLevel': dtype('int64'),
 'JobRole': dtype('O'),
 'JobSatisfaction': dtype('int64'),
 'MaritalStatus': dtype('O'),
 'MonthlyIncome': dtype('int64'),
 'MonthlyRate': dtype('int64'),
 'NumCompaniesWorked': dtype('int64'),
 'OverTime': dtype('O'),
 'PercentSalaryHike': dtype('int64'),
 'PerformanceRating': dtype('int64'),
 'RelationshipSatisfaction': dtype('int64'),
 'StockOptionLevel': dtype('int64'),
 'TotalWorkingYears': dtype('int64'),
 'TrainingTimesLastYear': dtype('int64'),
 'WorkLifeBalance': dtype('int64'),
 'YearsAtCompany': dtype('int64'),
 'YearsInCurrentRole': dtype('int64'),
 'YearsSinceLastPromotion': dtype('int64'),
 'YearsWithCurrManager': dtype('int64')}
```

In [51]:
```
#segregating the variables based on datatypes

numeric_variable_names  = [key for key in dict(employee_data_new.dtypes) if dict(em

categorical_variable_names = [key for key in dict(employee_data_new.dtypes) if dict
```

In [52]:
```
numeric_variable_names
```

Out[52]:
```
['Age',
 'DailyRate',
 'DistanceFromHome',
 'Education',
 'EnvironmentSatisfaction',
 'JobInvolvement',
 'JobLevel',
 'JobSatisfaction',
 'MonthlyIncome',
 'MonthlyRate',
 'NumCompaniesWorked',
 'PercentSalaryHike',
 'PerformanceRating',
 'RelationshipSatisfaction',
 'StockOptionLevel',
 'TotalWorkingYears',
 'TrainingTimesLastYear',
 'WorkLifeBalance',
 'YearsAtCompany',
 'YearsInCurrentRole',
 'YearsSinceLastPromotion',
 'YearsWithCurrManager']
```

In [53]:
```
categorical_variable_names
```

Out[53]:
```
['Attrition',
 'BusinessTravel',
 'Department',
 'EducationField',
 'JobRole',
 'MaritalStatus',
 'OverTime']
```

In [54]:
```
# store the numerical variables data in separate dataset

employee_data_num = employee_data_new[numeric_variable_names]
```

In [55]:
```
# store the categorical variables data in separate dataset

employee_data_cat = employee_data_new[categorical_variable_names]

#dropping the attrition
employee_data_cat.drop(["Attrition"], axis=1, inplace=True)
```

In [56]:
```
# converting into dummy variables

employee_data_cat = pd.get_dummies(employee_data_cat)
```

In [57]:
```
# merging the both numerical and categorical data

employee_data_final = pd.concat([employee_data_num, employee_data_cat,employee_data
```

In [58]:
```
employee_data_final.head()
```

Out[58]:

	Age	DailyRate	DistanceFromHome	Education	EnvironmentSatisfaction	JobInvolvement	JobL
0	41	1102	1	2	2	3	
1	49	279	8	1	3	2	
2	37	1373	2	2	4	2	
3	33	1392	3	4	4	3	
4	27	591	2	1	1	3	

5 rows × 49 columns

In [59]:
```
#final features
features = list(employee_data_final.columns.difference(["Attrition"]))
```

In [60]:
```
features
```

Out[60]:
```
['Age',
 'BusinessTravel_Non-Travel',
 'BusinessTravel_Travel_Frequently',
 'BusinessTravel_Travel_Rarely',
 'DailyRate',
 'Department_Human Resources',
 'Department_Research & Development',
 'Department_Sales',
 'DistanceFromHome',
 'Education',
 'EducationField_Human Resources',
 'EducationField_Life Sciences',
 'EducationField_Marketing',
 'EducationField_Medical',
 'EducationField_Other',
 'EducationField_Technical Degree',
 'EnvironmentSatisfaction',
 'JobInvolvement',
 'JobLevel',
 'JobRole_Healthcare Representative',
 'JobRole_Human Resources',
 'JobRole_Laboratory Technician',
 'JobRole_Manager',
 'JobRole_Manufacturing Director',
 'JobRole_Research Director',
 'JobRole_Research Scientist',
 'JobRole_Sales Executive',
 'JobRole_Sales Representative',
 'JobSatisfaction',
 'MaritalStatus_Divorced',
 'MaritalStatus_Married',
 'MaritalStatus_Single',
 'MonthlyIncome',
 'MonthlyRate',
 'NumCompaniesWorked',
 'OverTime_No',
 'OverTime_Yes',
 'PercentSalaryHike',
 'PerformanceRating',
 'RelationshipSatisfaction',
 'StockOptionLevel',
 'TotalWorkingYears',
 'TrainingTimesLastYear',
 'WorkLifeBalance',
 'YearsAtCompany',
 'YearsInCurrentRole',
 'YearsSinceLastPromotion',
 'YearsWithCurrManager']
```

In [ ]:
```
```

Separating the Target and the Predictors

In [61]:
```
# separating the target and predictors

X = employee_data_final[features]
y = employee_data_final[["Attrition"]]
```

In [62]:
```
X.shape
```

Out[62]:
```
(1470, 48)
```

Train_Test Split (Stratified Sampling of Y)

In [63]:
```
# Function for creating model pipelines
from sklearn.pipeline import make_pipeline

# function for crossvalidate score
from sklearn.model_selection import cross_validate

# to find the best
from sklearn.model_selection import GridSearchCV
```

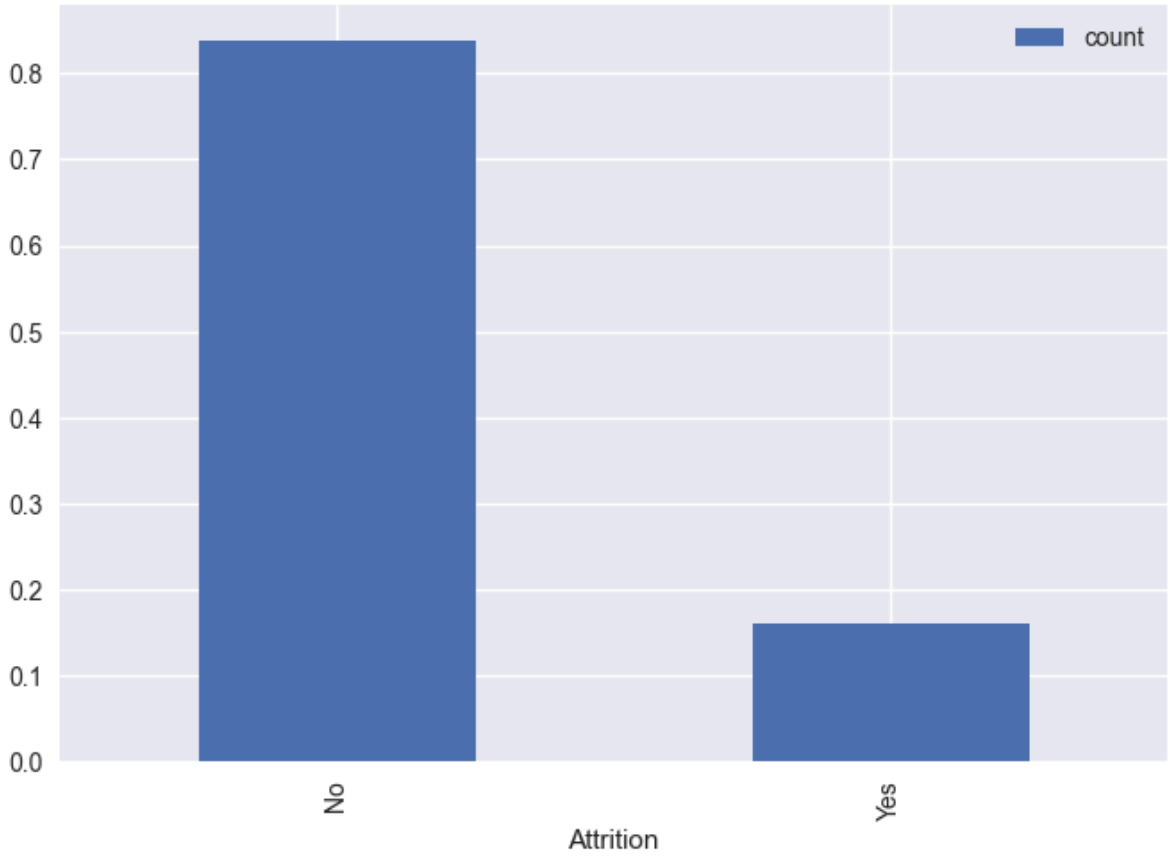
In [64]:
```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size= 0.3, stratify= y
```

In [65]:
```
# Checks
#proportion in training_data
y_train.Attrition.value_counts()/len(y_train)
```

Out[65]:
```
Attrition
No     0.838678
Yes    0.161322
Name: count, dtype: float64
```

In [66]:
```
#Checks
#Proportion in training data
pd.DataFrame(y_train.Attrition.value_counts()/len(y_train)).plot(kind ="bar")
```

Out[66]:
```
<Axes: xlabel='Attrition'>
```



In [67]:
```
#Proportion of test data
y_test.Attrition.value_counts()/len(y_test)
```

Out[67]:
```
Attrition
No     0.839002
Yes    0.160998
Name: count, dtype: float64
```

In [68]:
```
#make a pipeline for the decision tree model

pipelines = {
    "clf" : make_pipeline(DecisionTreeClassifier(max_depth=3, random_state=100))
}
```

Cross Validate

- To check the accuracy of the pipeline

In [69]:
```
scores = cross_validate(pipelines['clf'], X_train, y_train, return_train_score=True
```

In [70]:
```
scores['test_score'].mean()
```

Out[70]:
```
np.float64(0.8396590101823348)
```

Average accuracy of pipeline with Decision Tree Classifier is 83.48%

Cross-Validation and Hyper Parameters Tuning

Cross Validation is the process of finding the best combination of parameters for the model by traning and evaluating the model for each combination of the parameters

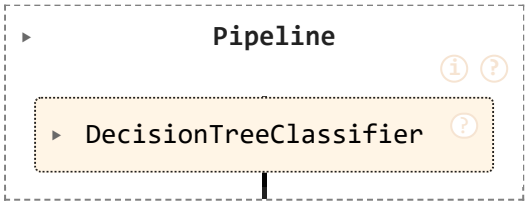
- Declare a hyper-parameters to fine tune the Decision Tree Classifier

Decision Tree is a greedy alogritum it searches the entire space of possible decision trees. so we need to find a optimum parameter(s) or criteria for stopping the decision tree at some point. We use the hyperparameters to prune the decision tree

In [71]:
```
decisiontree_hyperparameters = {
    "decisiontreeclassifier__max_depth": np.arange(3,12),
    "decisiontreeclassifier__max_features": np.arange(3,10),
    "decisiontreeclassifier__min_samples_split": [2,3,4,5,6,7,8,9,10,11,12,13,14,15
    "decisiontreeclassifier__min_samples_leaf" : np.arange(1,3)
}
```

In [72]:
```
pipelines['clf']
```

Out[72]:



Decision Tree Classifier with gini index

Fit and tune models with cross-validation

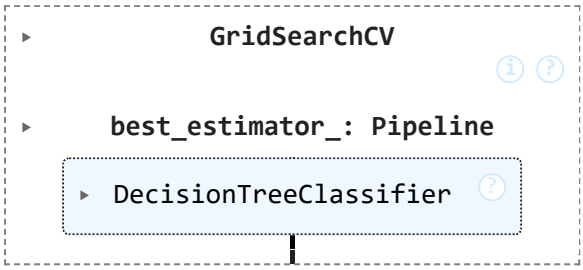
Now that we have our `pipelines` and `hyperparameters` dictionaries declared, we're ready to tune our models with cross-validation.

- We are doing 5 fold cross validation

In [73]:
```
#Create a cross validation object from decision tree classifier and it's hyperparam

clf_model = GridSearchCV(pipelines['clf'], decisiontree_hyperparameters, cv=5, n_jo
```

In [74]:
```
#fit the model with train data
clf_model.fit(X_train, y_train)
```

Out[74]:



In [75]:
```
#Display the best parameters for the Decison Tree Model
clf_model.best_params_
```

Out[75]:
```
{'decisiontreeclassifier__max_depth': np.int64(3),
 'decisiontreeclassifier__max_features': np.int64(7),
 'decisiontreeclassifier__min_samples_leaf': np.int64(1),
 'decisiontreeclassifier__min_samples_split': 10}
```

In [76]:
```
#Display the best score for the fitted model
clf_model.best_score_
```

Out[76]:
```
np.float64(0.8561780724603363)
```

In [77]:
```
#In Pipeline we can use the string names to get the decisiontree classifier
clf_model.best_estimator_.named_steps["decisiontreeclassifier"]
```

Out[77]:
```
DecisionTreeClassifier(max_depth=np.int64(3), max_features=np.int64(7),
                       min_samples_leaf=np.int64(1), min_samples_split=1
0,
                       random_state=100)
```

In [78]:
```
# saving into a variable to get graph
clf_best_model = clf_model.best_estimator_.named_steps["decisiontreeclassifier"]
```

Model Performance Evaluation

- On Test Data

In [79]:
```
#Making a dataframe of actual and predicted data from test set

tree_test_pred = pd.concat([y_test.reset_index(drop=True),pd.DataFrame(clf_model.pr
tree_test_pred.columns = ["actual","predicted"]

#setting the index to original index
tree_test_pred.index = y_test.index
```

In [80]:
```
tree_test_pred.head()
```

Out[80]:

	actual	predicted
34	Yes	Yes
1432	No	No
334	No	No
1068	Yes	No
736	No	No

In [81]:
```
#keeping only positive condition (yes for attrition)

pred_probability = pd.DataFrame(p[1] for p in clf_model.predict_proba(X_test))
pred_probability.columns = ["predicted_prob"]
pred_probability.index = y_test.index
```

In [82]:
```
# merging the predicted data and it's probability value

tree_test_pred = pd.concat([tree_test_pred,pred_probability],axis=1)
```

In [83]:
```
tree_test_pred.head()
```

Out[83]:

	actual	predicted	predicted_prob
34	Yes	Yes	0.632184
1432	No	No	0.220859
334	No	No	0.072165
1068	Yes	No	0.145985
736	No	No	0.072165

In [84]:
```
#converting the labels Yes --> 1 and No --> 0 for further operations below

tree_test_pred["actual_left"] = np.where(tree_test_pred["actual"] == "Yes",1,0)
tree_test_pred["predicted_left"] = np.where(tree_test_pred["predicted"] == "Yes",1,
```

In [85]:
```
tree_test_pred.head()
```

Out[85]:

	actual	predicted	predicted_prob	actual_left	predicted_left
34	Yes	Yes	0.632184	1	1
1432	No	No	0.220859	0	0
334	No	No	0.072165	0	0
1068	Yes	No	0.145985	1	0
736	No	No	0.072165	0	0

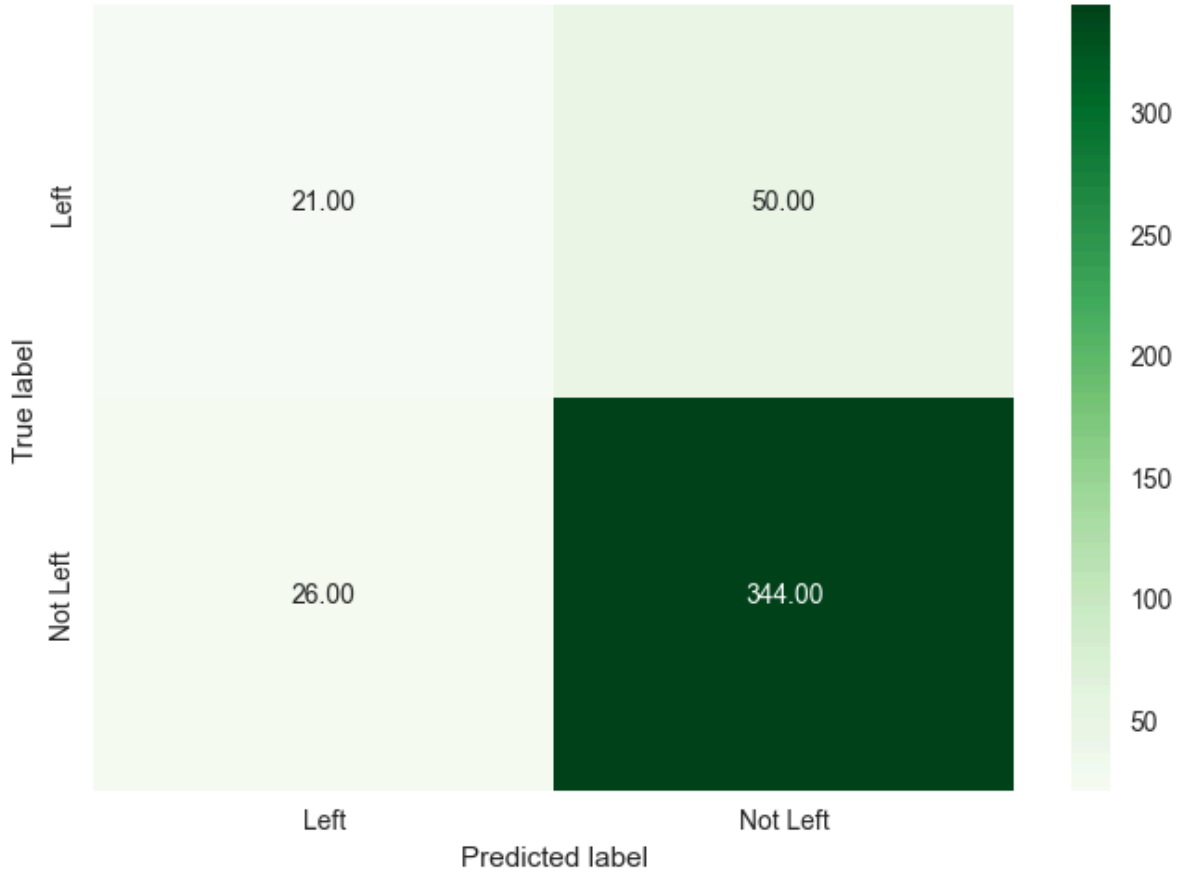
Confusion Matrix

The confusion matrix is a way of tabulating the number of misclassifications, i.e., the number of predicted classes which ended up in a wrong classification bin based on the true classes.

In [86]:
```
#confusion matrix
metrics.confusion_matrix(tree_test_pred.actual,tree_test_pred.predicted,labels=["Ye
```

Out[86]:
```
array([[ 21,  50],
       [ 26, 344]])
```

In [87]:
```
#confusion matrix visualisation using seaborn heatmap

sns.heatmap(metrics.confusion_matrix(tree_test_pred.actual, tree_test_pred.predicte
                                     labels=["Yes","No"]), cmap="Greens", annot=Tru
                                     xticklabels= ["Left","Not Left"], yticklabels=
plt.ylabel("True label")
plt.xlabel("Predicted label")
plt.show()
```

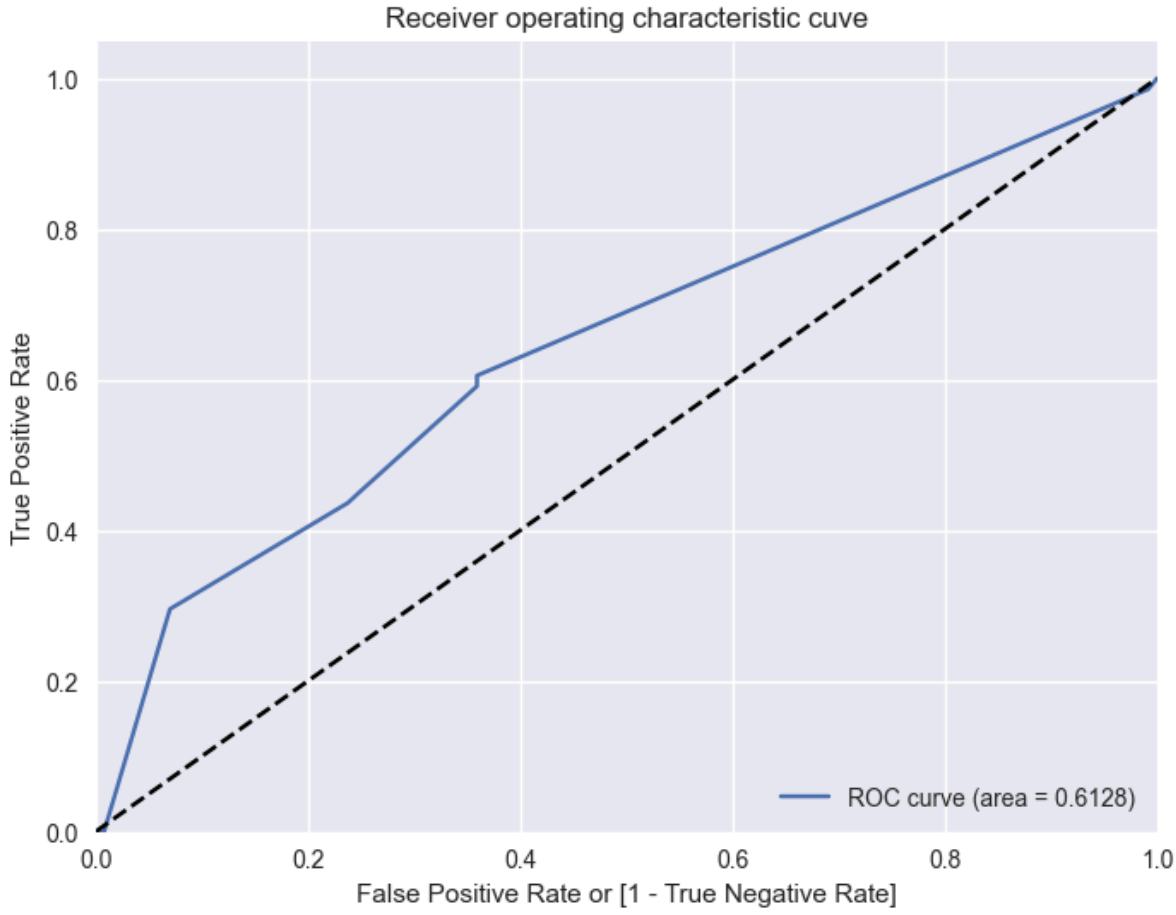


In [88]:
```
# Area Under ROC Curve
auc_score_test = metrics.roc_auc_score(tree_test_pred.actual_left,tree_test_pred.pr
```

```
print("AUROC Score :",round(auc_score_test,4))
```

```
AUROC Score : 0.6128
```

In [89]:
```
##Plotting the ROC Curve

fpr, tpr, thresholds = metrics.roc_curve(tree_test_pred.actual_left, tree_test_pred

plt.figure(figsize=(8, 6))
plt.plot( fpr, tpr, label='ROC curve (area = %0.4f)' % auc_score_test)
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate or [1 - True Negative Rate]')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic cuve')
plt.legend(loc="lower right")
plt.show()
```



From the ROC Curve, we have a choice to make depending on the value we place on true positive and tolerance for false positive rate

- If we wish to find the more people who are leaving, we could increase the true positive rate by adjusting the probability cutoff for classification. However by doing so would also increase the false positive rate. we need to find the optimum value of cutoff for classification

Metrics

- Recall: Ratio of the total number of correctly classified positive examples divide to the total number of positive examples. High Recall indicates the class is correctly recognized

- Precision: To get the value of precision we divide the total number of correctly classified positive examples by the total number of predicted positive examples. High Precision indicates an example labeled as positive is indeed positive

In [90]:
```
#calculating the recall score

print("Recall Score:",round(metrics.recall_score(tree_test_pred.actual_left,tree_te
```

```
Recall Score: 29.577
```

In [91]:
```
#calculating the precision score

print("Precision Score:",round(metrics.precision_score(tree_test_pred.actual_left,t
```

```
Precision Score: 44.681
```

In [92]:
```
print(metrics.classification_report(tree_test_pred.actual_left,tree_test_pred.predi
```

```
              precision    recall  f1-score   support

           0       0.87      0.93      0.90       370
           1       0.45      0.30      0.36        71

    accuracy                           0.83       441
   macro avg       0.66      0.61      0.63       441
weighted avg       0.80      0.83      0.81       441
```

Visualization of Decision Tree

- Dependencies
  - Need to install graphviz (conda install pydot graphviz)
  - Set the environment path variable to graphviz folder

In [93]:
```
# conda install pydot graphviz
# pip install pydotplus
```

In [94]:
```
from sklearn.tree import export_graphviz
```

In [95]:
```
import pydotplus as plot
```

In [96]:
```
from six import StringIO
from IPython.display import Image
from sklearn.tree import export_graphviz
import pydotplus as pdot
```

In [97]:
```
import os
os.environ["PATH"] += os.pathsep + "C:/Program Files/Graphviz/bin/"
```

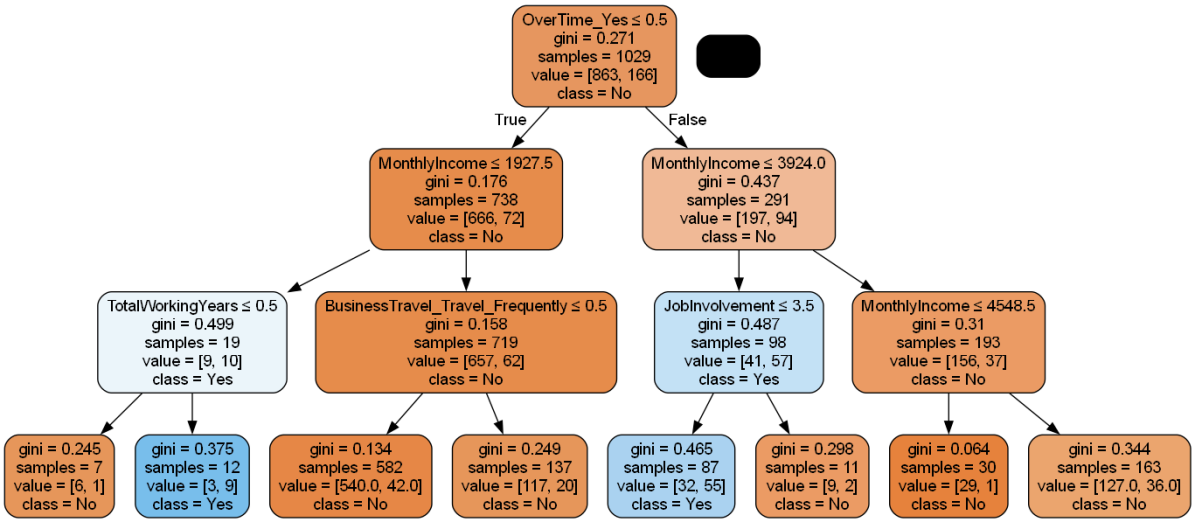
In [98]:
```
#write the dot data
dot_data = StringIO()
```

In [99]:
```
#export the decision tree along with the feature names into a dot file format

export_graphviz(clf_best_model,out_file=dot_data,filled=True,
                rounded=True,special_characters=True,feature_names = X_train.columr
```

In [100…
```
#make a graph from dot file
graph = pdot.graph_from_dot_data(dot_data.getvalue())
```

In [101…
```
Image(graph.create_png())
```

Out[101]:



In [102…
```
#export the tree diagram
graph.write_png("Employee_Attirtion.png")
```

Out[102]:
```
True
```

Conclusions

- The dependent variable for the decision tree is a OverTime which has two classes Yes or No.
- The most influential attribute to determine how to classify employee will leave or not is the Monthly Income variable.
- Even though the employee's having the average monthly income if their job involvement is average or less than average, those employees will most likely leave the company.
- Employee's whose monthly income is less than 2000 approximately will leave the company even if their total working years is more than 6 months.

Suggested Actions:

It's not sensible to focus on every employee who wants to leave because it costs time and energy for human resources management department. HR department need to focus on:

- Improving the work conditions

```
Provide an option for the employee's  to work from home, on a
flexible schedule, or in an office
with an ergonomic workspace, they will be more satisfied with their
work and more likely to
achieve a healthy work-life balance.
```

- Offer modest salaries and perks

To maintain the critical employee’s company need’s to offer equitable and modest salaries. You
can also give added perks like flexible schedules, travel discounts etc.

- Employee Engagement

When you have talented employee’s we need to find ways that you can help expand the
employee’s skill set, so that their involvement in the job increases. If their involvement is low,
they will get bored and think that they are not growing within the organization

In [ ]:

In [ ]: